GENSTOCKFOLIO / RAG DEMO

Hybrid Rag Solution

Earnings PDFs → indexed evidence → grounded answers

01

Ingestion pipeline

02

Retrieval pipeline

03

Generation pipeline

ARCHITECTURE & PIPELINE WALKTHROUGH

Hybrid Rag Solution

SECTION 01 / GENSTOCKFOLIO

Ingestion pipeline

PDF parsing, KPI extraction, chunk preparation, and indexing.

Flutter FastAPI LangChain Chroma

From an earnings PDF to a saved report

Client → upload → validation → parsing → extraction → chunk preparation → persistence.

01–03 · CLIENT & SERVER

FLUTTER CLIENT — 01

Choose an earnings PDF

PDF bytes + ticker / period
Flutter file picker
Submit for ingestion

FASTAPI SERVER — 02

Receive the upload

POST /api/v1/ingestion/upload
multipart/form-data
Save PDF in data/uploads/

PYTHON + SQLITE — 03

Validate & fingerprint

Check %PDF- header + SHA-256
SQLite duplicate lookup
force=true skips cache reuse

PARSER

PYPDF · PYTHON — 04

Read page-aligned text

pypdf.PdfReader
Keep 1-based page numbers
Track blank pages

PYTHON CLASSIFIER — 05

Tag financial sections

Rules score headers + keywords
Cash flow, profitability, etc.
Retain page-domain metadata

EXTRACTOR

OPENAI — 06

Extract structured KPIs

gpt-4o-mini via LangChain
Selected financial pages
Pydantic schema + math rules

CHUNKER

LANGCHAIN — 07a

Recursive Chunker

1,000 chars · 100 overlap · non-blank pages

PYTHON — 07b

Metadata Enrichment

Ticker, period, page, domain + chunk ID

08 · PERSISTENCE

SQLITE · SQLALCHEMY — 08

Save report & KPIs

SQLite: stock_analyst.db
documents + financial_kpis
Persisted before indexing

CONTINUE ON SLIDE 02 → After saving, return the result and prepare the search indexes independently (proposed).

Return results while indexing continues

Proposed flow · after saving report + KPIs, fork into a response path and independent background indexing.

CONTINUED FROM SLIDE 01

SQLITE · SQLALCHEMY 08

Save report & KPIs

SQLite: stock_analyst.db
documents + financial_kpis
Persisted before indexing

RESPONSE NOW

Return without waiting for the search indexes.

FASTAPI RESPONSE 09

Return the result

Persisted report + KPIs
Response does not wait for indexing
Background work continues

FLUTTER + RIVERPOD 10

Refresh the dashboard

QuarterlyKpis.fromJson
Riverpod addAndSelectReport
Render scorecards + charts

BACKGROUND INDEXER

Independent execution · no join back to the response.

OPENAI + CHROMA 11

Embed & store in Chroma

text-embedding-3-small · 512-dim
LangChain Documents + metadata
Chroma: earnings_chunks

LANGCHAIN · BM25 12

Build the BM25 index

BM25Retriever.from_documents
Keyword ranking; no embeddings
Corpus: data/bm25/documents.json

PROPOSED FLOW · Current code still indexes before returning; independent background execution requires a server change.

What makes ingestion useful for RAG?

Keep small, traceable pieces of evidence that the vector and keyword indexes can search.

PARSER

PYPDF · PYTHON — 01

Keep page boundaries

pypdf extracts page text
Page numbers start at 1
Blank pages are skipped later

PYTHON CLASSIFIER — 02

Keep financial context

Rule-based domain labels
Headers + body keywords
Each page keeps its domain

EXTRACTOR

OPENAI · GPT-4O-MINI — 03

Resolve report identity

KPIs supply canonical metadata
Ticker + fiscal quarter + year
Statement pages sent to LLM

CHUNKER

LANGCHAIN — 04

Recursive Chunker

RecursiveCharacterTextSplitter
1,000 characters per chunk
100-character overlap

PYTHON ENRICHMENT — 05

Metadata Enrichment

Stable chunk ID + text
Ticker, period, page + domain
Chunk position + char count

Chunking uses all non-blank pages, including narrative pages outside the KPI extraction subset.

A SAVED CHUNK · actual inspection-artifact metadata

```
chunk_id: "TSLA_Q2_2026_p3_c0"
ticker / period: "TSLA" / "Q2" / 2026
page_number: 3
financial_domain: "cash_flow"
chunk position: 0 of 4 · char_count: 943
```

WHERE THE EVIDENCE LIVES TODAY

Chunks feed Chroma + BM25

03b_enriched_chunks_*.json / .md

Written when save_artifacts=true

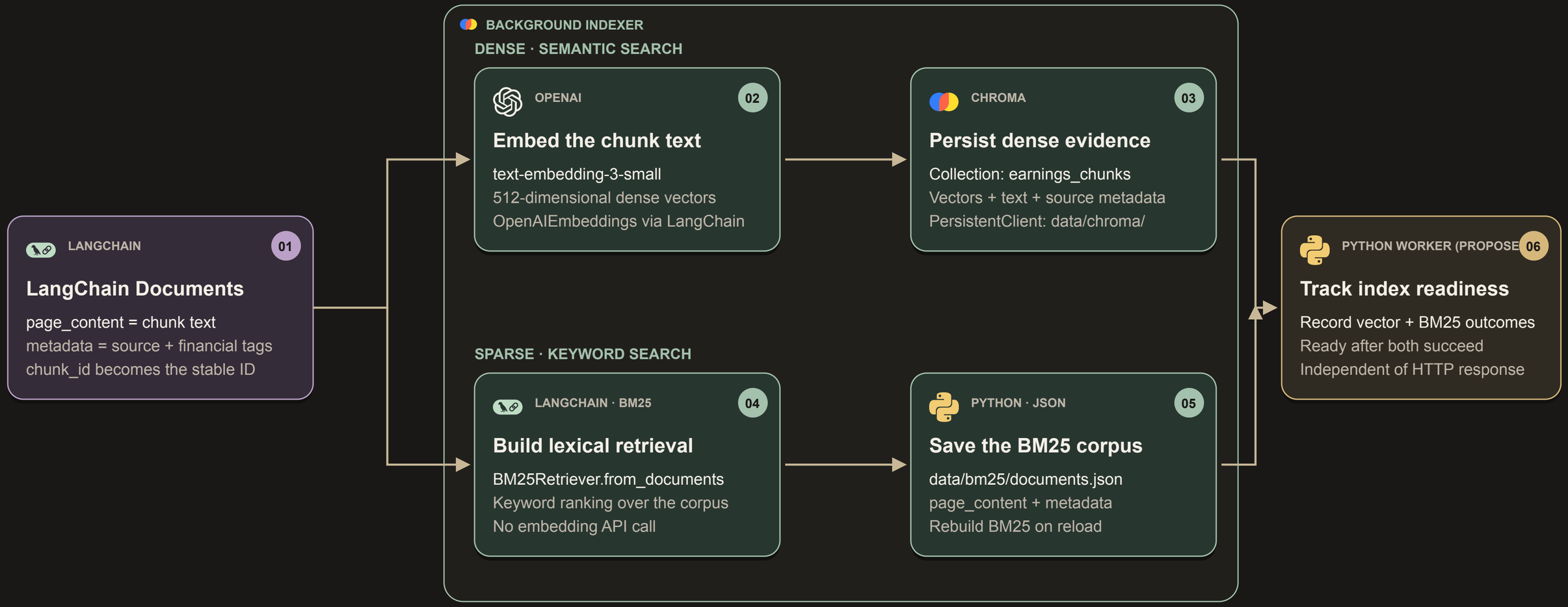
The Flutter upload enables artifact saving.

INDEXING HANDOFF · implemented in the ingestion orchestrator

Same enriched chunks → OpenAI embeddings + Chroma dense index, and LangChain BM25 sparse index

Background indexing after persistence

Proposed worker path · the saved report result is already returned while the indexes are being prepared.



Report availability and RAG search readiness are separate: the dashboard can load while indexing continues.

Vector and BM25 work may remain sequential inside the worker; neither blocks the response branch.

Proposed orchestration. Current code performs both indexing attempts inline before returning the result.

Hybrid Rag Solution

SECTION 02 / GENSTOCKFOLIO

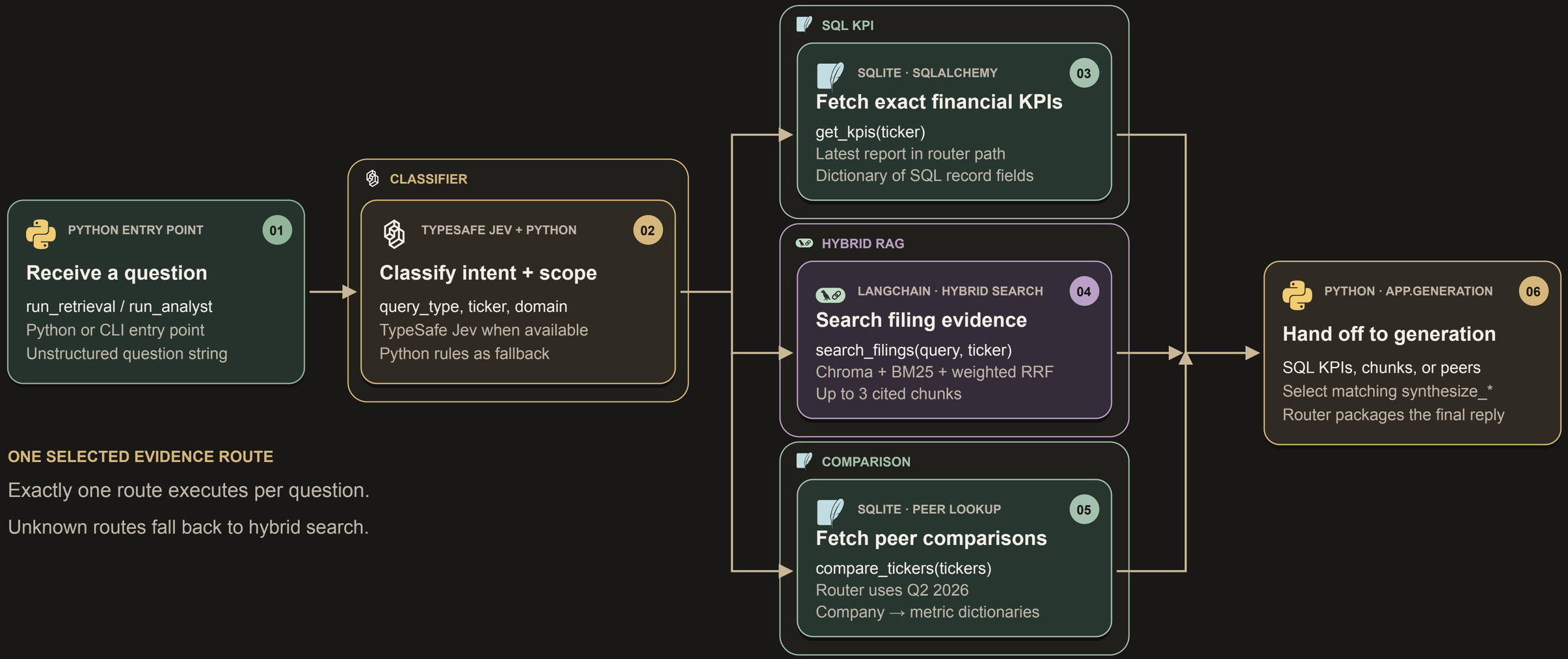
Retrieval pipeline

Intent routing, financial lookups, and hybrid evidence search.

TypeSafe Chroma LangChain

One question, three evidence routes

Classify intent, select one retrieval tool, and pass its evidence to the matching generation function.



ONE SELECTED EVIDENCE ROUTE

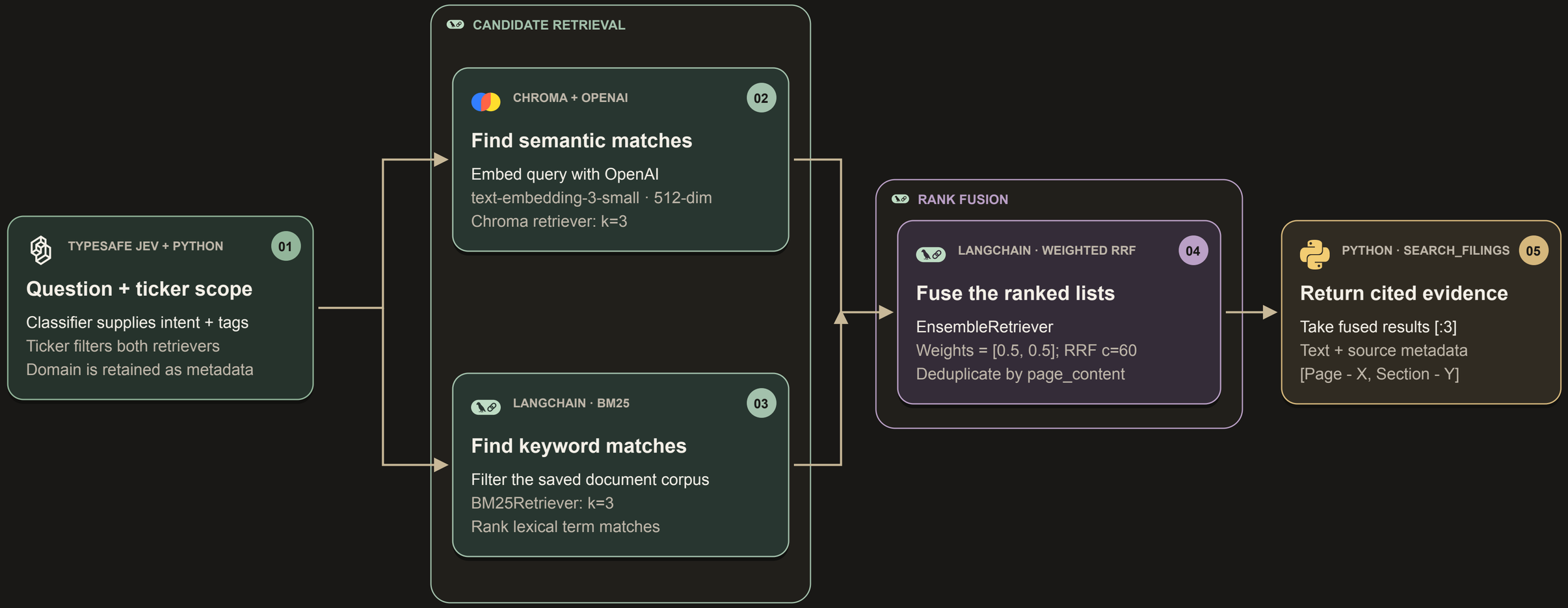
Exactly one route executes per question.

Unknown routes fall back to hybrid search.

Current interface: backend Python / CLI. Flutter chat still generates local responses; no chat HTTP or SSE route is wired.

Hybrid retrieval: meaning + exact keywords

The analyst route asks each retriever for 3 candidates, fuses rankings, and returns up to 3 evidence chunks.



FILTERS & FALLBACK

Normal hybrid path filters ticker only. On engine.search failure, BM25 fallback also applies financial_domain if provided.

No cross-encoder reranker is called. Direct HybridEngine.search defaults to k=5; the analyst router overrides it to k=3.

Hybrid Rag Solution

SECTION 03 / GENSTOCKFOLIO

Generation pipeline

Retrieved evidence, grounded answers, and source citations.

LangChain

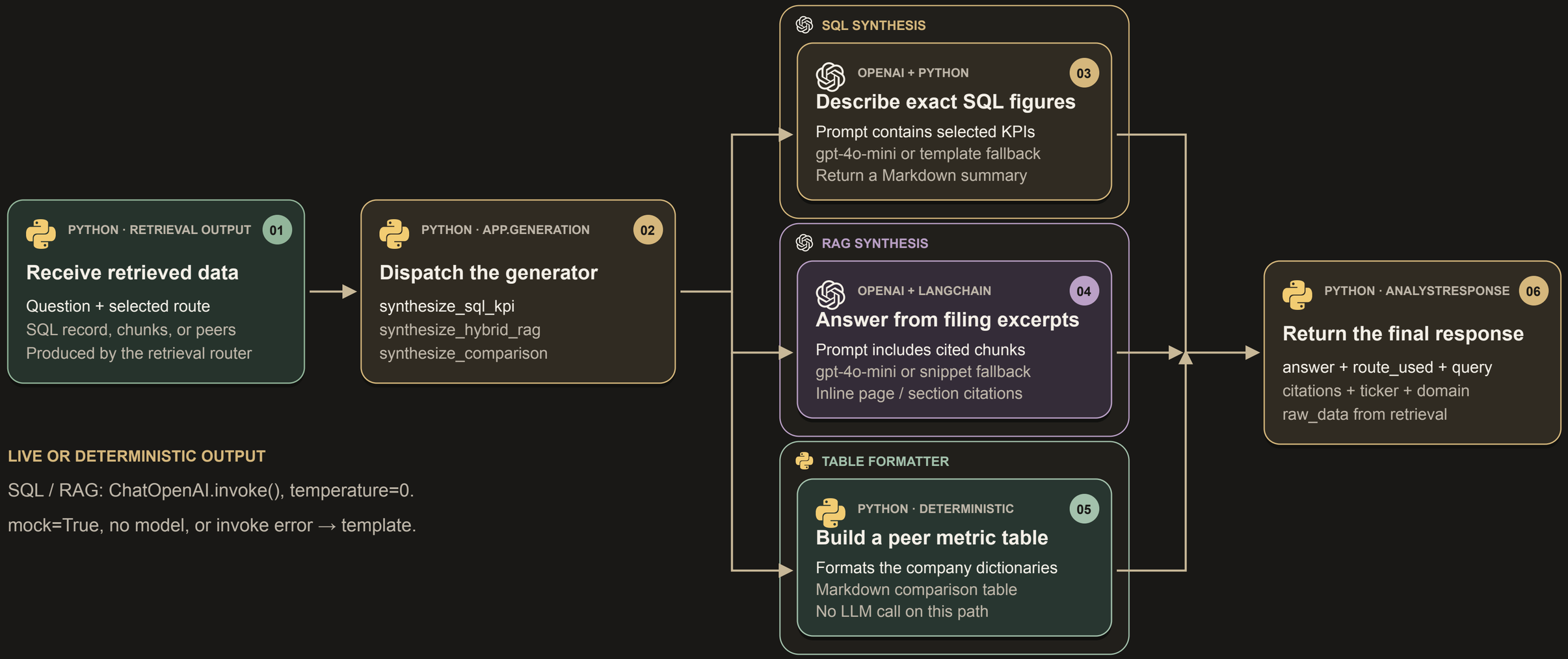
OpenAI

Python

02 DIAGRAMS / PIPELINE WALKTHROUGH

From retrieved evidence to an answer

Each retrieval route selects its own synthesis function; peer comparison always uses a deterministic table.



LIVE OR DETERMINISTIC OUTPUT

SQL / RAG: ChatOpenAI.invoke(), temperature=0.

mock=True, no model, or invoke error → template.

The functions return answer strings. The retrieval router adds metadata; no token streaming or citation validator runs here.

RAG generation: keep evidence in the prompt

The live narrative path uses retrieved excerpts and inline citation labels to guide a grounded answer.

PROMPT & MODEL

ANSWER & RESPONSE

PYTHON · RETRIEVAL 01

Receive cited chunks

query + retrieved chunk payloads
text, page, domain, citation
Empty input → no-evidence notice

LANGCHAIN · MESSAGES 02

Assemble the prompt

Question + Source [citation]
Join excerpts with separators
Use ONLY supplied evidence

OPENAI 03

Generate the answer

ChatOpenAI: gpt-4o-mini
temperature=0.0; invoke()
System + Human messages

PYTHON · GENERATION 04

Return model content

resp.content becomes answer
No post-generation validation
Complete string, no SSE tokens

PYTHON · ROUTER 05

Package the response

Router adds the retrieved chunks
route_used = hybrid_rag
AnalystResponse to Python caller

CITATION INSTRUCTION

Every key factual claim should cite its excerpt:
[Page - X, Section - Y]
The prompt requests this format; code does not verify
citation correctness or detect unsupported claims.

DETERMINISTIC FALLBACK

Used for mock mode, missing model, or invoke failure.
Format the first 280 characters of each excerpt
as a bullet, retaining its citation label.
This is excerpt formatting, not a live LLM answer.

The comparison path bypasses this LLM flow entirely. SQL synthesis uses a separate exact-figures prompt.

The backend result is available through Python / CLI; Flutter's current chat widget is not connected to these functions.